UNIVERSIDAD NACIONAL DE INGENIERÍA

FACULTAD DE INGENIERÍA DE SISTEMAS

DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SOFTWARE Y SISTEMAS

INFORME TÉCNICO DE INGENIERÍA DE CALIDAD

Diseño, Implementación y Verificación Formal de la Suite de Automatización de Pruebas con PHPUnit 10.5

Aplicación al Sistema Transaccional de Facturación Electrónica SUNAT UBL 2.1

Curso: Aseguramiento de la Calidad de Software (QA)

Docente Encargado: Equipo de Cátedra de Calidad de Software

Integrantes del Equipo de Trabajo:

- Apolinario Rodriguez, Luis Fernando *(Líder de Calidad & Expositor)*
- Umeres Vilca, Gustavo Aron *(Arquitectura de Software & Tests)*
- Vasquez Falcón, Mariana Ysabel *(Dominio Fiscal & Casos de Uso)*
- Ventocilla Alcantara, Alessandro Tito *(Criptografía UBL 2.1 & Mocks)*

Lima, Perú

Noviembre de 2026

Resumen Ejecutivo

El presente documento constituye el informe técnico y académico del proyecto de aseguramiento de la calidad de software aplicado al Sistema de Facturación Electrónica bajo el estándar UBL 2.1, regulado por la Superintendencia Nacional de Aduanas y de Administración Tributaria (SUNAT) del Perú.

En los sistemas fiscales y financieros, el margen de error tolerado es nulo: una discrepancia de un céntimo en el redondeo monetario, una alteración en la firma digital o un fallo de consistencia en el inventario deriva en el rechazo inmediato del comprobante mediante Códigos de Retorno (CDR), la nulidad tributaria de la transacción y sanciones pecuniarias de hasta una Unidad Impositiva Tributaria (UIT). Para erradicar este riesgo operativo, se diseñó e implementó una suite integral de 39 pruebas automatizadas y 140 aserciones rigurosas bajo el framework PHPUnit 10.5 y PHP 8.2.

La estrategia de pruebas adopta la metodología de Desarrollo Guiado por Pruebas (*Test-Driven Development* – TDD), técnicas de diseño formal por Partición de Equivalencia y Análisis de Valores Límite (*Boundary Value Analysis* – BVA), el patrón canónico *Arrange-Act-Assert* (AAA) y los principios FIRST. Se implementó una base de datos SQLite puramente en memoria RAM (`:memory:`) para pruebas de persistencia e idempotencia concurrente, y se emplearon Dobles de Prueba (*Mocks*) para aislar completamente las dependencias de red de los servicios SOAP de SUNAT. La suite completa se ejecuta en 0.21 segundos con 0 fallos y 0 errores (100 % verde), demostrando la solidez y viabilidad industrial de la arquitectura.

Índice

1. Introducción y Marco Normativo Fiscal **3**

1.1. Contexto de la Facturación Electrónica en el Perú 3

1.2. El Estándar Internacional UBL 2.1 (Universal Business Language) 3

1.3. Criptografía XMLDSIG y Firma Digital X.509 3

1.4. Problemática Operativa y Riesgos de Calidad 3

2. Arquitectura del Sistema y Diseño de Software **4**

2.1. Estructura en Capas Desacopladas 4

2.2. Objetos de Valor Inmutables (Value Objects) 4

2.3. El Algoritmo Oficial de SUNAT Módulo 11 4

2.4. Motor Tributario y Régimen Multitasa (`CalculadoraTributaria`) 5

3. Estrategia de Aseguramiento de la Calidad (QA) **5**

3.1. Metodología de Desarrollo Guiado por Pruebas (TDD) 5

3.2. Técnicas Formales de Diseño de Pruebas 5

3.2.1. Partición de Equivalencia (Equivalence Partitioning) 5

3.2.2. Análisis de Valores Límite (Boundary Value Analysis – BVA) 6

3.3. Principios FIRST en Pruebas Unitarias 6

3.4. Rigor en Aserciones: `assertSame` vs `assertEquals` 6
3.5. Pruebas Parametrizadas: Atributo nativo PHP 8 DataProvider 6
3.6. Dobles de Prueba: Aislamiento del Web Service SOAP de SUNAT 7

4. Pirámide de Pruebas y Matriz Integral de Cobertura **7**
4.1. Distribución de Pruebas según Mike Cohn 7
4.2. Matriz Oficial de Cobertura de las 39 Pruebas 7

5. Auditoría de Casos de Prueba Críticos (Código Fuente) **7**
5.1. Caso 1: Algoritmo SUNAT Módulo 11 en Value Object `Ruc` 7
5.2. Caso 2: Liquidación de Comprobante Mixto Multirégimen e ICBPER 8
5.3. Caso 3: Prueba de Seguridad Criptográfica (Tamper Test Código 2335) 9
5.4. Caso 4: Persistencia Concurrente e Idempotencia (Doble Clic) 9
5.5. Caso 5: Resiliencia Transaccional y Rollback Automático de Kardex 9

6. Guía de Reproducción, Ejecución y Certificación Empírica **10**
6.1. Instrucciones para Ejecución en Terminal 10
6.2. Métricas Oficiales de Ejecución 10

7. Conclusiones y Recomendaciones de Ingeniería **10**
7.1. Conclusiones 10
7.2. Recomendaciones 11

Referencias Bibliográficas y Normativas **11**

1. Introducción y Marco Normativo Fiscal

1.1. Contexto de la Facturación Electrónica en el Perú

La obligatoriedad de la emisión electrónica de comprobantes de pago en el Perú surge al amparo del Decreto Ley N° 25632 y las sucesivas Resoluciones de Superintendencia emitidas por la SUNAT (notoriamente las Resoluciones N° 097-2012/SUNAT, 117-2017/SUNAT y modificatorias). Estas resoluciones estipulan que toda transacción comercial que implique entrega de bienes o prestación de servicios debe respaldarse mediante un comprobante digital estructurado que goce de validez legal, certeza temporal y no repudio.

1.2. El Estándar Internacional UBL 2.1 (Universal Business Language)

El formato oficial exigido para facturas (*Invoice*), boletas de venta (*Bill*), notas de crédito y notas de débito es UBL 2.1, basado en esquemas XML estandarizados por OASIS. El XML tributario exige una jerarquía rigurosa de nodos canónicos, entre los que destacan:

- `cac:AccountingSupplierParty`: Identificación unívoca del emisor fiscal (RUC y razón social).
- `cac:AccountingCustomerParty`: Identificación del receptor comercial (RUC, DNI o Clientes Varios).
- `cac:TaxTotal`: Desglose tributario exacto discriminado por Código de Tipo de Tributo SUNAT (Catálogo N° 05: 1000 para IGV, 7152 para ICBPER).
- `cac:LegalMonetaryTotal`: Importes netos, descuentos globales y monto total a cobrar expresado en moneda de curso legal (PEN o USD).

1.3. Criptografía XMLDSIG y Firma Digital X.509

La validez jurídica de un comprobante radica en su integridad criptográfica. El emisor debe firmar el documento digitalmente utilizando un certificado digital X.509 v3 autorizado por INDECOPI. El proceso contempla:

1. **Canonicalización (C14N):** Conversión del documento XML a una representación canónica estándar (omitiendo espacios arbitrarios y ordenando atributos) según la recomendación W3C.
2. **Cálculo del Resumen Criptográfico (Digest):** Aplicación del algoritmo criptográfico unidireccional SHA-256 sobre el cuerpo del documento (`DigestValue`).
3. **Firma Asimétrica (SignatureValue):** Cifrado del digest empleando la clave privada RSA (mínimo 2048 bits) del certificado digital.

Cualquier intento de alteración posterior a la firma, aun cuando implique un solo dígito en el monto a pagar, invalida el *digest* y es calificado por SUNAT como adulteración tributaria mediante el **Código de Rechazo 2335** (*"El documento ha sido alterado o el hash no coincide"*).

1.4. Problemática Operativa y Riesgos de Calidad

En el desarrollo de software tributario, los defectos se traducen en contingencias financieras y penales severas:

- **Discrepancias de Redondeo Decimal:** Cálculos intermedios con punto flotante primitivo (`float`) generan desfases de fracciones de céntimo frente a las sumatorias exigidas por el algoritmo de validación de SUNAT.

- **Fallas en Doble Emisión (Idempotencia):** Intermitencias en las redes de telecomunicaciones provocan que cajeros o terminales de punto de venta envíen dos veces la misma solicitud de cobro, generando duplicación de débitos fiscales.
- **Inconsistencia de Inventario ante Rechazos:** Si el sistema descuenta existencias físicas en el almacén pero el servidor de SUNAT rechaza la factura (por ejemplo, con Código 2800 si el cliente tiene condición de no habido o baja tributaria), se genera una discrepancia grave entre el inventario físico y el Kardex contable.

2. Arquitectura del Sistema y Diseño de Software

Para garantizar alta testeabilidad, mantenimiento evolutivo e independencia tecnológica, el sistema adopta los principios de la **Arquitectura Hexagonal** (*Ports and Adapters*) y diseño guiado por el dominio (*Domain-Driven Design* – DDD).

2.1. Estructura en Capas Desacopladas

El código se organiza en tres capas estrictamente delimitadas:

1. **Capa de Dominio (Domain):** Núcleo puro del negocio. Contiene las reglas tributarias peruanas, los algoritmos matemáticos y las entidades y *Value Objects*. No posee dependencias externas ni conoce marcos de trabajo web ni motores de persistencia.
2. **Capa de Aplicación (Application):** Coordina el flujo transaccional. Contiene los Casos de Uso (como `EmitirFacturaUseCase`), define las interfaces de los puertos de salida (repositorios, emisores SOAP) y gestiona la atomicidad de las operaciones.
3. **Capa de Infraestructura (Infrastructure):** Implementa los adaptadores concretos: repositorios sobre base de datos SQLite, constructores de XML UBL 2.1, firmadores criptográficos OpenSSL y clientes de comunicación SOAP.

2.2. Objetos de Valor Inmutables (Value Objects)

Se prohíbe el uso de tipos primitivos sueltos (*Primitive Obsession*) para conceptos críticos del dominio:

- `Ruc`**:** Encapsula la cadena del RUC, validando en su constructor la longitud de 11 dígitos, el prefijo tributario válido (10 para personas naturales, 20 para personas jurídicas) y la exactitud matemática del dígito verificador.
- `MontoMonetario`**:** Modela valores monetarios representados internamente como cadenas numéricas con dos decimales exactos (*Fixed-Point BCMath*), impidiendo el uso de flotantes binarios IEEE 754.

2.3. El Algoritmo Oficial de SUNAT Módulo 11

La autenticidad de los 11 dígitos del RUC responde a un algoritmo de control ponderado por residuos. Sea el vector de dígitos del RUC $D = [d_1, d_2, \ldots, d_{11}]$ y el vector oficial de factores ponderadores asignados por SUNAT:

$$F = [5, 4, 3, 2, 7, 6, 5, 4, 3, 2] \tag{1}$$

El procedimiento de verificación computacional se define en las siguientes etapas:

1. Producto escalar de los primeros 10 dígitos por los factores de ponderación:

$$S = \sum_{i=1}^{10} d_i \cdot f_i \tag{2}$$

2. Cálculo del residuo de la división entera por módulo 11 y sustracción de base:

$$R = 11 - (S \pmod{11}) \tag{3}$$

3. Asignación del dígito verificador esperado DV:

$$DV = \begin{cases} 0 & \text{si } R = 10 \\ 1 & \text{si } R = 11 \\ R & \text{en cualquier otro caso} \end{cases} \tag{4}$$

4. Condición de validez: $DV = d_{11}$. Si la igualdad no se cumple, el objeto rechaza la construcción lanzando una `DomainValidationException`.

2.4. Motor Tributario y Régimen Multitasa (`CalculadoraTributaria`)

El motor tributario calcula en tiempo real las bases imponibles y los tributos de comprobantes complejos según el catálogo de tipos de afectación al IGV (Catálogo N° 07 de SUNAT):

- **Operación Onerosa Gravada (Código 10):** Aplica la tasa general del 18 % sobre el valor de venta base: IGV $= V_v \times 0{,}18$.
- **Operación Exonerada (Código 20):** Aplica para productos de canasta básica y libros. IGV $= 0{,}00$; no se traslada débito fiscal.
- **Operación Inafecta (Código 30):** Servicios médicos, exportaciones y operaciones fuera del campo de aplicación tributaria.
- **Impuesto a las Bolsas Plásticas (ICBPER - Código 7152):** Tasa fija de S/ 0.50 por bolsa plástica entregada. La regla tributaria exige que el ICBPER **no se incorpore a la base imponible del IGV**, sino que se sume directamente al importe total a liquidar.

3. Estrategia de Aseguramiento de la Calidad (QA)

3.1. Metodología de Desarrollo Guiado por Pruebas (TDD)

El desarrollo de las reglas de liquidación y los Value Objects se ejecutó siguiendo el ciclo canónico *Red - Green - Refactor*:

1. **Fase Roja (Red):** Redacción del caso de prueba formal antes de implementar la clase o método, estableciendo la aserción esperada según el manual técnico de SUNAT. La ejecución inicial falla de forma predecible.
2. **Fase Verde (Green):** Escritura del código mínimo necesario en el dominio para satisfacer la aserción y lograr que la prueba pase a verde.
3. **Fase de Refactorización (Refactor):** Optimización de la estructura de código, eliminación de duplicidades, introducción de Value Objects e inmutabilidad, garantizando que la suite continúe en verde.

3.2. Técnicas Formales de Diseño de Pruebas

3.2.1. Partición de Equivalencia (Equivalence Partitioning)

Permite clasificar el espacio de entradas en subconjuntos donde el comportamiento del sistema es previsiblemente uniforme:

- *Clase Válida 1:* RUC con longitud exacta de 11 caracteres y residuo módulo 11 exacto.
- *Clase Inválida 1:* Cadenas alfanuméricas o longitud menor a 11 dígitos.
- *Clase Inválida 2:* Cadenas con longitud mayor a 11 dígitos.
- *Clase Inválida 3:* 11 dígitos numéricos cuyo dígito verificador discrepa del residuo ponderado.

3.2.2. Análisis de Valores Límite (Boundary Value Analysis – BVA)

La experiencia en aseguramiento de calidad evidencia que la mayor densidad de fallas radica en las fronteras de decisión.

- **Regla Fiscal de Boleta Mayor a S/ 700.00:** El marco tributario de SUNAT establece que si una boleta de venta supera los S/ 700.00, es obligatorio identificar al cliente con su número de documento (DNI o Carné de Extranjería). Por debajo o igual a este monto, se autoriza la emisión a "Clientes Varios".
- **Puntos de Frontera Auditados:**
  - *Límite inferior (*$700{,}00 - 0{,}01 = S/\ 699{,}99$*):* Válido sin DNI.
  - *Valor en frontera (S/* 700,00*):* Válido sin DNI.
  - *Límite superior (*$700{,}00 + 0{,}01 = S/\ 700{,}01$*):* **Rechazo inmediato** con excepción de negocio si se envía cliente anónimo.

3.3. Principios FIRST en Pruebas Unitarias

Las pruebas automatizadas del proyecto satisfacen plenamente los 5 postulados FIRST:

- **Fast (Rápidas):** La suite completa de 39 tests se ejecuta en 0.21 segundos en memoria RAM.
- **Independent (Aisladas):** Cada test inicializa y destruye su propio estado en el ciclo `setUp()` y `tearDown()`, sin depender del orden de ejecución.
- **Repeatable (Repetibles):** El resultado es determinista y no depende de conexiones de red ni variables ambientales del host.
- **Self-validating (Auto-evaluables):** La prueba reporta un veredicto binario inequívoco: éxito (verde) o fallo (rojo).
- **Timely (Oportunas):** Construidas en conjunto con el desarrollo de la lógica de dominio.

3.4. Rigor en Aserciones: `assertSame` vs `assertEquals`

En sistemas financieros, la igualdad débil (`==`) constituye un defecto de diseño crítico. En PHP, la expresión `assertEquals(100, "100.00")` resulta verdadera debido a la conversión automática de tipos (*type juggling*).

La suite impone el uso estricto de `assertSame()` (`===`), que audita de forma simultánea el **tipo de dato** y el **contenido exacto**. Esto asegura que los totales formateados para el XML no presenten truncamiento de ceros a la derecha y se mantengan como cadenas formales con precisión de 2 dígitos decimales.

3.5. Pruebas Parametrizadas: Atributo nativo PHP 8 DataProvider

En versiones anteriores de PHPUnit, las pruebas parametrizadas dependían de comentarios interpretados en tiempo de ejecución. En PHPUnit 10.5 y PHP 8.2, se adopta el atributo nativo fuertemente tipado:

```
#[DataProvider('proveedorRucsValidos')]
public function testAceptaRucsValidosConModulo11(string $numero): void
{
    $ruc = new Ruc($numero);
    $this->assertSame($numero, $ruc->getNumero());
}
```

Listing 1: Uso de atributos nativos de PHP 8 para pruebas parametrizadas.

3.6. Dobles de Prueba: Aislamiento del Web Service SOAP de SUNAT

Para someter a prueba los casos de uso transaccionales sin depender de los servidores externos de la administración tributaria, se implementaron dobles de prueba (*Test Doubles*) conforme a la taxonomía de Martin Fowler. Se generaron **Mocks** de la interfaz `SunatSenderInterface`, configurando expectativas exactas sobre el número de invocaciones (`expects($this->once())`) y simulando respuestas síncronas de CDR de éxito (Código 0) y de rechazo (Código 2800).

4. Pirámide de Pruebas y Matriz Integral de Cobertura

4.1. Distribución de Pruebas según Mike Cohn

La arquitectura de aseguramiento de calidad distribuye las 39 pruebas del proyecto siguiendo el modelo de la pirámide de pruebas de Mike Cohn:

- **Base – Pruebas Unitarias de Dominio (26 Tests / 67 %):** Ejecutadas en microsegundos. Validan algoritmos matemáticos, Value Objects y el motor tributario.
- **Nivel Medio – Pruebas de Integración (7 Tests / 18 %):** Validan la generación de XML UBL 2.1, la firma digital XMLDSIG SHA-256 y la persistencia atómica en SQLite.
- **Cúspide – Pruebas de Feature y Casos de Uso (6 Tests / 15 %):** Validan el flujo transaccional completo, la idempotencia contra reintentos y la restitución de inventario en Kardex ante rechazos simulados de SUNAT.

4.2. Matriz Oficial de Cobertura de las 39 Pruebas

En la Tabla 1 se detalla la matriz exhaustiva de las pruebas automatizadas verificadas en el sistema.

5. Auditoría de Casos de Prueba Críticos (Código Fuente)

A continuación se analizan los cinco casos de prueba de mayor trascendencia técnica y operativa del sistema.

5.1. Caso 1: Algoritmo SUNAT Módulo 11 en Value Object `Ruc`

Garantiza que ningún comprobante sea generado con un RUC sintáctica o matemáticamente inválido.

```
1  private function validarModulo11(string $ruc): void
2  {
3      $factores = [5, 4, 3, 2, 7, 6, 5, 4, 3, 2];
4      $suma = 0;
5      for ($i = 0; $i < 10; $i++) {
6          $suma += ((int) $ruc[$i]) * $factores[$i];
7      }
8      $resto = 11 - ($suma % 11);
9      $digitoVerificador = match ($resto) {
10         10 => 0,
11         11 => 1,
12         default => $resto
13     };
14     if ($digitoVerificador !== ((int) $ruc[10])) {
15         throw new DomainValidationException(
16             "El digito verificador del RUC es invalido"
17         );
18     }
19 }
```

Listing 2: Implementación del algoritmo Módulo 11 en `src/Domain/ValueObjects/Ruc.php`.

Tabla 1: Matriz oficial de cobertura de pruebas automatizadas (PHPUnit 10.5).

Capa de Software	Componentes Auditados & Reglas Fiscales	Tests	Aserciones
Dominio Fiscal	`Ruc.php`: Algoritmo Módulo 11 oficial SUNAT	8	26
Dominio Fiscal	`Ruc.php`: Validación de longitudes y caracteres	4	14
Dominio Fiscal	`MontoMonetario.php`: Formateo decimal y suma exacta	2	8
Motor Tributario	`CalculadoraTributaria.php`: IGV 18 % sobre gravados	4	16
Motor Tributario	`CalculadoraTributaria.php`: Exonerados e Inafectos	3	10
Motor Tributario	`CalculadoraTributaria.php`: Bolsas ICBPER S/ 0.50	2	8
Motor Tributario	`CalculadoraTributaria.php`: Comprobante Mixto Complejo	3	10
Criptografía & XML	`UblBuilder`: Generación de nodos UBL 2.1	2	8
Criptografía & XML	`XmlVerifier`: Detección de adulteración (Tamper 2335)	2	8
Persistencia	`SqliteRepo`: Inserción y consulta en memoria	1	4
Persistencia	`SqliteRepo`: Idempotencia transaccional (doble clic)	2	8
Casos de Uso	`EmitirFacturaUseCase`: Flujo exitoso con Mock SUNAT	2	6
Casos de Uso	`EmitirFacturaUseCase`: Rechazo 2800 y Rollback Kardex	2	8
Casos de Uso	`EmitirFacturaUseCase`: Validación BVA Boleta ¿S/ 700	2	6
Totales Auditados	**Suite completa ejecutada con éxito (100 % verde)**	**39**	**140**

5.2. Caso 2: Liquidación de Comprobante Mixto Multirégimen e ICBPER

Evalúa que las bolsas fiscales no se contaminen recíprocamente: el IGV se calcula exclusivamente sobre las operaciones gravadas, mientras que el ICBPER se añade al importe total a pagar sin inflar el valor neto de venta.

```
1  public function testCalculaComprobanteMixtoComplejo(): void
2  {
3      $items = [
4          new ItemComprobante(1, 100.00, TipoAfectacion::GRAVADO_OP_ONEROSA),
5          new ItemComprobante(1, 50.00,
               TipoAfectacion::EXONERADO_OPERACION_ONEROSA),
6          new ItemComprobante(2, 0.50, TipoAfectacion::GRAVADO_OP_ONEROSA,
               icbper: 1.00)
7      ];
8
9      $totales = $this->calculadora->calcular($items);
10
11     $this->assertSame('101.00', $totales->totalGravado);
12     $this->assertSame('50.00', $totales->totalExonerado);
13     $this->assertSame('18.18', $totales->totalIgv);
14     $this->assertSame('1.00', $totales->totalIcbper);
15     $this->assertSame('170.18', $totales->importeTotal);
16 }
```

Listing 3: Prueba unitaria en `tests/Unit/CalculadoraTributariaTest.php`.

5.3. Caso 3: Prueba de Seguridad Criptográfica (Tamper Test Código 2335)

Verifica que la alteración de un solo byte en el archivo XML posterior a la firma sea detectada inmediatamente mediante la canonicalización y el recálculo del digest SHA-256.

```
public function testDetectaAdulteracionDeMontoEnXmlFirmado(): void
{
    $xmlOriginal = $this->builder->generarFacturaXml($this->factura);
    $xmlFirmado = $this->firmador->firmarXml($xmlOriginal);

    // Simulacion de ataque en transito: alterar de S/ 100.00 a S/ 10.00
    $xmlAdulterado = str_replace(
        '<cbc:PayableAmount currencyID="PEN">100.00</cbc:PayableAmount>',
        '<cbc:PayableAmount currencyID="PEN">10.00</cbc:PayableAmount>',
        $xmlFirmado
    );

    $this->expectException(XmlTamperDetectedException::class);
    $this->expectExceptionCode(2335); // Codigo oficial SUNAT
    $this->verificador->verificarDigest($xmlAdulterado);
}
```

Listing 4: Prueba de seguridad en `tests/Integration/XmlAndCryptoIntegrationTest.php`.

5.4. Caso 4: Persistencia Concurrente e Idempotencia (Doble Clic)

Demuestra que peticiones idénticas originadas por intermitencias de red o repulsación del botón de cobro devuelven la misma transacción almacenada sin generar dos comprobantes fiscales en la base de datos.

```
public function testPrevieneDobleEmisionPorIdempotencia(): void
{
    $claveIdempotencia = 'req-uuid-test-12345';

    // Primer clic
    $res1 = $this->service->procesar($this->solicitud, $claveIdempotencia);
    $this->assertSame('COMPLETED', $res1->estado);

    // Segundo clic inmediato con la misma clave
    $res2 = $this->service->procesar($this->solicitud, $claveIdempotencia);

    $this->assertSame($res1->numeroComprobante, $res2->numeroComprobante);
    $this->assertSame(1, $this->repo->contarComprobantes());
}
```

Listing 5: Test de idempotencia en `tests/Integration/SqlitePersistenceIntegrationTest.php`.

5.5. Caso 5: Resiliencia Transaccional y Rollback Automático de Kardex

Verifica la preservación de la invariante de inventario: si SUNAT emite un CDR de rechazo fiscal (Código 2800), el stock previamente reservado de productos se restituye de forma atómica en el almacén.

```
public function testRollbackDeKardexAnteRechazoFiscalSunat(): void
{
    $sunatMock = $this->createMock(SunatSenderInterface::class);
    $sunatMock->method('sendBill')->willReturn(new SunatResponse(
        codigo: '2800',
        descripcion: 'El contribuyente se encuentra con Estado: BAJA'
    ));

```

```
9       $this->expectException(SunatRechazoException::class);
10      $emisor = new EmitirFacturaUseCase($this->repo, $sunatMock,
            $this->kardex);
11
12      // Intento de emitir factura descontando 5 unidades
13      $emisor->emitir($this->solicitudCon5Unidades);
14
15      // Verificacion de invariante: el stock debe permanecer intacto en 15
16      $stockActual = $this->kardex->obtenerStock('PROD-001');
17      $this->assertSame(15, $stockActual);
18  }
```

Listing 6: Prueba de feature en `tests/Feature/FacturacionElectronicaMocksFeatureTest.php`.

6. Guía de Reproducción, Ejecución y Certificación Empírica

6.1. Instrucciones para Ejecución en Terminal

Para reproducir la totalidad de las aserciones en un entorno limpio de desarrollo, se ejecutan las siguientes instrucciones en terminal (PowerShell o Bash):

```
1 # 1. Posicionarse en el directorio del backend PHP
2 cd Billing/backend
3
4 # 2. Descargar dependencias e inicializar el autoloader de Composer
5 composer install --prefer-dist --no-interaction
6
7 # 3. Ejecutar la suite completa con reporte detallado
8 vendor/bin/phpunit --testdox --colors=never
```

Listing 7: Comandos para clonación y ejecución de la suite de pruebas.

6.2. Métricas Oficiales de Ejecución

La ejecución de la suite arrojó los siguientes indicadores empíricos oficiales:

- **Total de Pruebas Ejecutadas:** 39 casos de prueba automatizados.
- **Aserciones Evaluadas:** 140 aserciones estrictas.
- **Tasa de Aprobación:** 100 % (0 fallos, 0 errores, 0 riesgos, 0 advertencias).
- **Tiempo Total de Ejecución:** 0.21 segundos.
- **Consumo de Memoria RAM:** 12.00 MB.

7. Conclusiones y Recomendaciones de Ingeniería

7.1. Conclusiones

1. **Eficacia del Desacoplamiento Arquitectónico:** La Arquitectura Hexagonal y la disciplina DDD permitieron someter a prueba el 100 % de las complejas reglas tributarias peruanas en memoria RAM, erradicando la dependencia de servidores externos inestables durante el ciclo de pruebas.
2. **Erradicación de Riesgos Financieros con Tipado Estricto:** El empleo sistemático de `assertSame()` en lugar de `assertEquals()` protegió al sistema contra desbordamientos decimales y conversiones de tipo implícitas que habrían ocasionado el rechazo sistemático de comprobantes por parte de SUNAT.

3. **Resiliencia Transaccional e Integridad Operativa:** Las pruebas de persistencia e integración demostraron que la combinación de claves de idempotencia y transacciones atómicas de Kardex asegura la integridad contable del negocio frente a anomalías de red o pulsaciones dobles de cajero.
4. **Aceleración del Ciclo de Retroalimentación:** La ejecución de 39 pruebas en 0.21 segundos permite la integración de la suite en canales de Integración Continua (CI/CD) antes de cada despliegue a producción.

7.2. Recomendaciones

- Extender la batería de pruebas de integración para cubrir Guías de Remisión Electrónica (GRE) y Comprobantes de Retención/Percepción (UBL 2.0 / 2.1).
- Incorporar pruebas de mutación (*Infection PHP*) para medir la resiliencia y exhaustividad de las 140 aserciones implementadas.
- Mantener la política de Cero Tolerancia a tipos flotantes (`float`) en cualquier cálculo de naturaleza fiscal.

Referencias Bibliográficas y Normativas

1. OASIS. (2013). *Universal Business Language (UBL) v2.1 OASIS Standard*. OASIS Open.
2. SUNAT. (2023). *Manual de Autorización de Comprobantes de Pago Electrónicos UBL 2.1 y Catálogos de Códigos Tributarios*. Superintendencia Nacional de Aduanas y de Administración Tributaria, Lima, Perú.
3. Bergmann, S. (2024). *PHPUnit 10 Manual: Automated Testing for PHP*. Sebastian Bergmann Open Source Projects.
4. Fowler, M. (2007). *Mocks Aren't Stubs*. MartinFowler.com.
5. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
6. Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley.
7. IEEE. (2013). *ISO/IEC/IEEE 29119 Software and Systems Engineering – Software Testing*. International Organization for Standardization.